

Comparative Analysis: AgentAuth vs. Entra Agent ID (Gateway & OBO)

This document provides a technical comparison between **AgentAuth** (this project) and the **Microsoft Entra Agent ID Gateway Architecture** (including the OBO Flow), as implemented in the entra-agent-id-agw repository.

This document provides a technical comparison between **AgentAuth** (this project) and the **Microsoft Entra Agent ID On-Behalf-Of (OBO) Flow** as described in the OBO Guide.

1. Executive Summary

Feature	Entra Agent ID (Gateway/OBO)	AgentAuth (GAuth)
Architectural Model	Infrastructure-Centric. Relies on Azure “Blueprints” and centralized IdP (Entra ID) for identity definition.	Protocol-Centric. Relies on cryptographic key pairs and Power of Attorney (PoA) chains for identity and delegation.
Trust Model	Centralized Trust. Trust flows from the IdP (Entra) minting tokens (T1, T2).	Decentralized Trust. Trust flows from the Principal’s signature on the PoA and local verification.
Runtime Dependency	Online with Azure. Requires connectivity to Entra ID for OBO exchange and Blueprint validation.	Offline-Capable. Verification is local using public keys; no centralized runtime dependency.
Agent Identity	Implicit / Metadata. Defined by “Blueprint” registration in Entra and associated Service Principal.	Explicit / Attested. Defined by cryptographic keys and signed AttestationProof (Human vs. AI distinction).
Delegation Policy	Coarse-Grained. OAuth Scopes (e.g., User.Read) defined in Azure App Registrations.	Fine-Grained. Value limits, time windows, and tool restrictions defined in the PoA itself.

2. Architecture Drill-Down

A. The Delegation Flow

Entra Agent ID (Standard OBO) The flow depends on a **Token Exchange** pattern (RFC 8693 style): 1. **User Authentication**: User gets a token Tc (User Token). 2. **Agent Identity**: Agent service authenticates to Entra to get T1 (Agent Token). 3. **Exchange**: Agent sends Tc + T1 to Entra ID. 4. **Result**: Entra validates both and issues T2 (Access Token) with the user's identity but "acting as" the agent. 5. **Consumption**: Resource API validates T2 (signature from Entra).

AgentAuth (GAuth) The flow depends on **Cryptographic Chaining** and **Attestation**: 1. **Delegation (PoA)**: Principal (User) signs a **Power of Attorney** object (JSON/CBOR) granting specific rights to the Agent's public key. 2. **Agent Integrity**: Agent authenticates via its private key (Ed25519/BLS). The **VerificationService** validates the agent's standing via **CommercialRegisterService**. 3. **Invocation**: Agent presents the **PoA Chain + Request Signature** directly to the Resource (or Sidecar). 4. **Verification**: The Resource (via AgentAuth Policy Engine) validates the signature chain locally. No "exchange" call to a central server is required at runtime.

B. Identity Provisioning & Definition

Entra Agent ID ("Blueprints") Identity is defined via **Infrastructure Configuration**: * **Blueprints**: Created via PowerShell/Graph API in Azure. * **Service Principals**: The runtime identity is an Azure Service Principal associated with the Blueprint. * **Constraint**: Tightly coupled to the Azure control plane. An agent *is* an Azure object.

AgentAuth ("Attestation") Identity is defined via **Cryptographic Attestation**: * **Key Generation**: The agent generates its own Key Pair (Ed25519/BLS). * **Attestation**: The agent submits proof of its configuration (model hash, safety capabilities) to an **AttestationAuthority**. * **Result**: A signed **AttestationProof** (Phase 11) is issued. The agent *is* a key pair with a certified safety rating. * **Advantage**: Portable across clouds, on-prem, and edge devices.

C. Gateway vs. Sidecar Patterns

Entra AGW (Gateway Pattern)

- **Focus**: Bridging legacy apps to Entra ID.
- **Mechanism**: A Gateway (or Sidecar) intercepts traffic, performs the OBO exchange with Entra, and injects the resulting T2 token.
- **Goal**: App code behaves as if the user called it directly; the "Agent" nature is abstracted by the infra.

AgentAuth (Verification Service)

- **Focus:** Enforcing complex delegations.
- **Mechanism:** The `VerificationService` (embedded library or microservice) inspects the *entire delegation chain*.
- **Goal:** The App acts on the specific *constraints* (e.g., “Authorized for \$50 only”). The “Agent” nature is explicit and critical for liability tracking.

C. Offline vs. Online

- **Entra: Online-Critical.** If Entra ID is down or unreachable, the OBO exchange fails. The agent cannot act.
- **GAUTH: Offline-Capable.** Once the PoA is issued, the agent can act anywhere (even air-gapped) as long as the resource has the trust anchors (root keys). Revocation checks can be batched or decentralized (OCSP-style).

3. Integration Strategy

AgentAuth is designed to **complement** OBO flows where finer control is needed:

1. **GAUTH as the Policy Engine:** Use AgentAuth to generate the “Permission” signal.
 - *Scenario:* User delegates complex rights via GAUTH PoA.
 - *Bridge:* The Agent presents the GAUTH PoA to an **Identity Assertion Service** (RFC 7523), which validates the fine-grained policy and *then* mints an OBO token (T2) compatible with legacy apps.
2. **GAUTH as the “Agent ID” Provider:** The `AttestationProof` generated by AgentAuth (proving the agent is a valid, compliant AI) can be embedded into the `client_assertion` used in the OBO exchange.

4. Conclusion

The Entra OBO pattern is excellent for **enterprise identity propagation** within the Microsoft ecosystem. **AgentAuth** provides the **legal and operational layer** required for autonomous agents: * **Legal Validity** (PoA/Fiduciary tracking). * **Safety Constraints** (Model limits, granular scopes). * **Resilience** (Decentralized verification).

Recommendation: Use **Entra OBO** for service-to-service auth transport. Use **AgentAuth** to govern *what* the agent is actually allowed to do within that pipe.